

# Game Objects

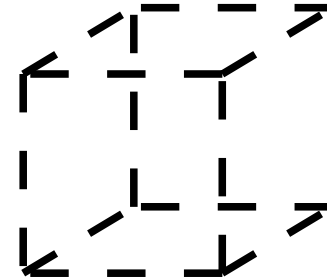
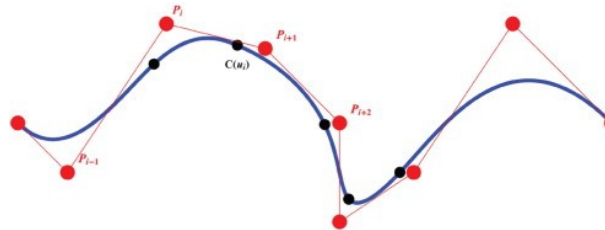
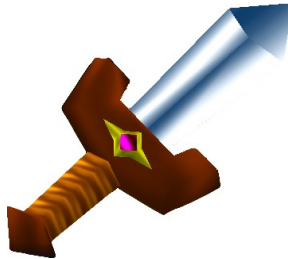
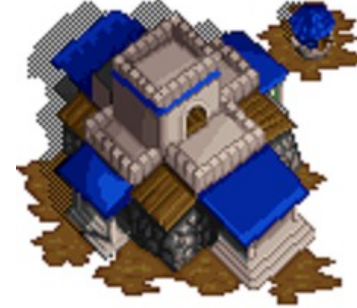


Image sources: <https://toraiinxamikaze.deviantart.com/art/Halo-2-Master-Chief-251551553>, <https://www.planetminecraft.com/project/minecraft-antigravity-portal-gun-test-chambers-plus-mods/>, [https://aminoapps.com/c/mario/page/item/whats-in-the-mystery-block/D8oi\\_NI2aRQkozWaapp45q41Rk3MGv6](https://aminoapps.com/c/mario/page/item/whats-in-the-mystery-block/D8oi_NI2aRQkozWaapp45q41Rk3MGv6), [https://wow.gamepedia.com/Town\\_Hall\\_\(Warcraft\\_II\)](https://wow.gamepedia.com/Town_Hall_(Warcraft_II)), [http://zelda.wikia.com/wiki/File:Kokiri\\_Sword\\_\(Ocarina\\_of\\_Time\).png](http://zelda.wikia.com/wiki/File:Kokiri_Sword_(Ocarina_of_Time).png), <https://designrfix.com/inspiration/inspiration-beautifully-rendered-3d-vehicles>

B-Spline-de-3er-grado-La-posicion-del-nodo-depende-de-los-puntos-de-control\_fig7\_262518034

# What are Game Objects? (1/3)

---



- Most video games take place in a two- or three-dimensional virtual game world
- A game world is typically comprised of numerous discrete elements
- These elements fall into two categories:
  - Static elements
    - Terrain, buildings, roads, bridges, ...
    - Anything that doesn't move or interact with gameplay in an active way
  - Dynamic elements
    - Characters, vehicles, weaponry, floating power-ups and health packs, collectible objects, particle emitters, dynamic lights, invisible regions used to detect important events in the game, splines that define the paths of objects, ...

# What are Game Objects? (2/3)



- A game world is comprised of both static and dynamic elements



Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

# What are Game Objects? (3/3)

---



- Gameplay is concentrated within the dynamic elements of a game
- The layout of the static background plays a crucial role as well
  - E.g. Cover-based shooter played in a big empty room,
  - Platformer game without any holes or static platforms, ...
- States of the dynamic elements, changes over time
  - Location, orientation and internal states
  - Updated by various gameplay systems
- The term game state refers to the current state of all dynamic game world elements, taken as a whole.
- The ratio of dynamic to static elements varies from game to game
  - Most 3D games: relatively small number of dynamic elements
  - Dynamic elements are more expensive in terms of CPU resources
  - The higher the ratio of dynamic to static elements, the more “alive” the game world can seem to the player

# Static vs. Dynamic Elements

---



- Distinction between the dynamic and static elements is a bit blurry
  - Background objects may be animated (waterfalls, plants, ...)
  - Different game engines draw different lines between static and dynamic elements
    - Everything is potentially a dynamic element
  - Distinction serves primarily as an optimization tool
    - Vertices of a static triangle mesh can be specified in world space
    - Lighting can be precomputed, in the form of static vertex lighting, light maps, shadow maps, static ambient occlusion information
    - Virtually anything can be precomputed for static objects!
  - Static elements can exist in multiple versions
    - E.g. Destructible environment with (undamaged, damaged, destroyed)
    - Can be swapped dynamically during a certain event (explosion, ...)

# Dynamic Elements: Game Objects

---



- Dynamic elements of a game are usually designed in an object-oriented fashion
- This approach is intuitive and natural for game designers and programmers alike
- We'll use the term game object (GO) to refer to virtually any dynamic element within a game world.
  - Game objects are commonly referred to as entities, actors, agents, ...
- Game object is essentially a collection of
  - attributes (the current state of the object) and
  - behaviors (how the state changes over time and in response to events)
- Classified by type: different attribute schemes and behaviors
- Instances of a particular type share the same attribute schema and the same set of behaviors, but the values of the attributes differ from instance to instance.

# Distinction between type and instance



- Pac-Man

- Game object types:

- Ghost
    - Pellet
    - Power pill
    - Fruit
    - Pac-Man

- Game object instances:

- Up to four instances of the type “ghost”
    - 50–100 instances of the type “pellet”
    - Four “power pill” instances
    - Up to two instances of the type “fruit”
    - One instance of the “Pac-Man” type

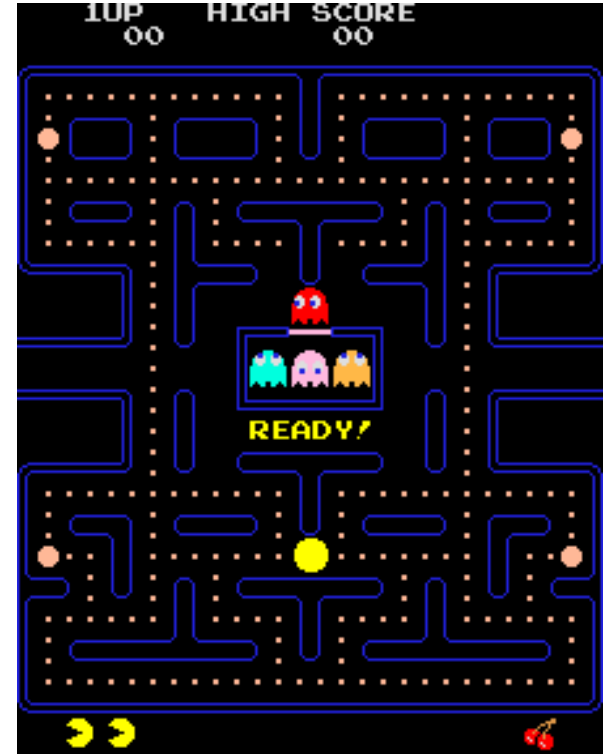
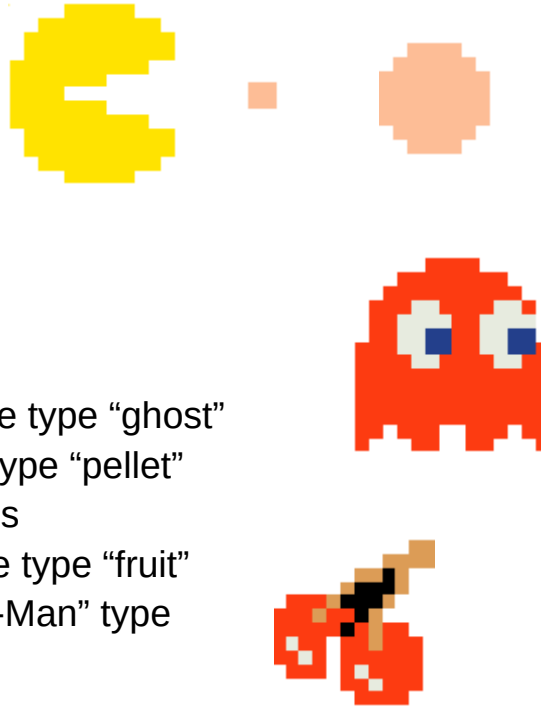


Image sources: <https://en.wikipedia.org/wiki/Pac-Man>, <https://forums.nesdev.com/viewtopic.php?f=5&t=12543>

# Tool-Side Design vs. Runtime Design

---



- A game really has two distinct but closely interrelated object models:
  - The tool-side object model is defined by the set of game object types seen by the designers within the world editor.
  - The runtime object model is defined by whatever set of language constructs and software systems the programmers have used to implement the tool-side object model at runtime.
- These models might be identical or entirely different
- Virtually all game engines have some form of tool-side object model and a corresponding runtime implementation of that object model
- We will focus on the runtime object model!



# Runtime Object Model Architectures

---



- Most game engines follow one of two basic architectural styles:
  - Object-centric
    - Each game object is represented by a single class instance or a small collection of interconnected instances
    - Each object has a set of attributes and behaviours that are encapsulated within its class (or classes)
    - The game world is just a collection of game objects
  - Property-centric
    - Each game object is represented only by a unique id
    - The properties of each game object are distributed across many data tables, one per property type, and keyed by object id
    - The properties themselves are often implemented as instances of hard-coded classes
    - The behavior of a game object is implicitly defined by the collection of properties from which it is composed.

# Object-Centric Architectures (1/10)



- Monolithic Class Hierarchies (1/2)
  - A class hierarchy is the most intuitive and straightforward way to represent a collection of interrelated game object types.
  - Typically a common root class called `GameObject`, for example
  - common, generic functionality tends to exist at the root of the hierarchy
  - classes toward the leaves of the hierarchy tend to add increasingly specific functionality

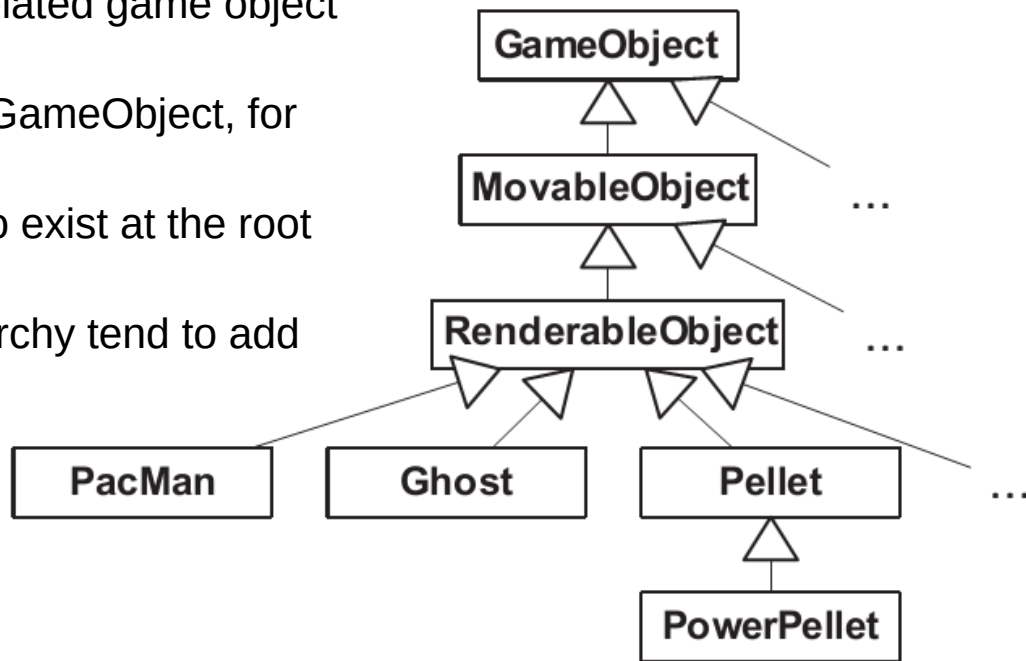


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

# Object-Centric Architectures (2/10)



- Monolithic Class Hierarchies (2/2)
  - Class hierarchy usually begins small and simple
  - As class hierarchies grow, they have a tendency to deepen and widen simultaneously
  - Virtually all classes in the game object model inherit from a single, common base class

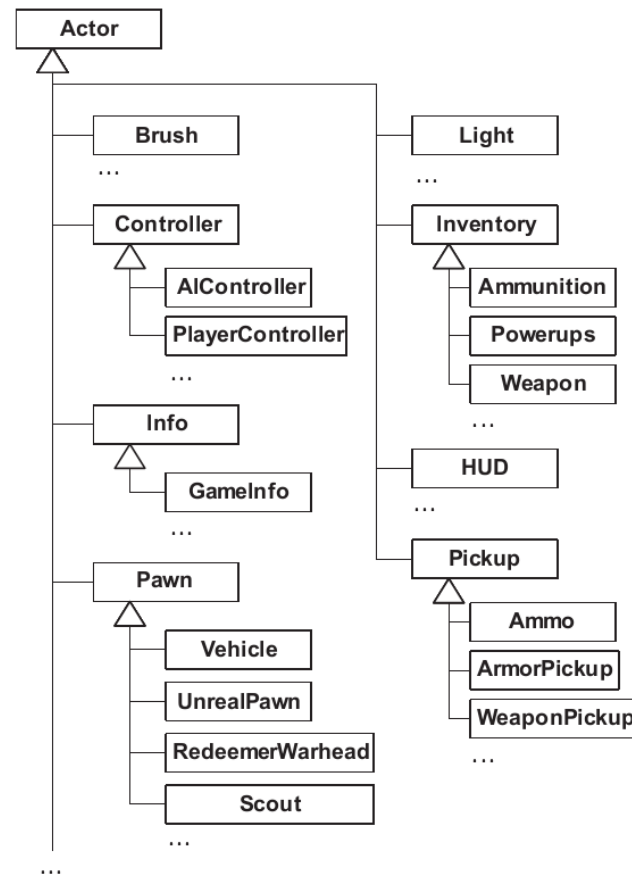


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

# Object-Centric Architectures (3/10)



- Problems with Deep, Wide Hierarchies
  - Understanding, Maintaining, and Modifying Classes
    - To understand a class, you need to understand all of its parent classes!
    - Virtual functions are especially problematic
  - Inability to Describe Multidimensional Taxonomies
    - Hierarchies can only classify objects along a single “axis” - according to one particular set of criteria - at each level of the tree.
    - The class hierarchy can not define a new type of object whose characteristics were not anticipated when the hierarchy was first designed

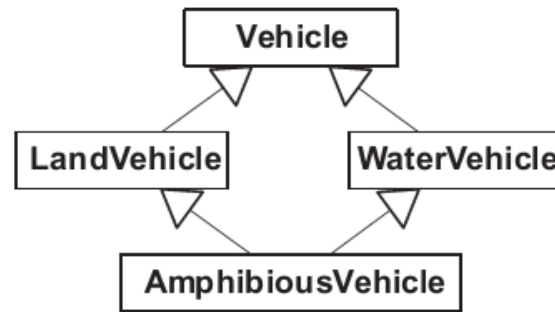
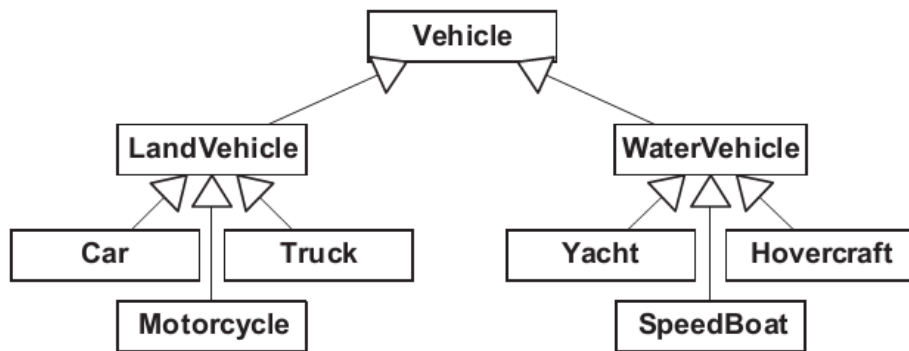


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

# Object-Centric Architectures (4/10)



- Multiple Inheritance: The Deadly Diamond
  - Can lead to an object that contains multiple copies of its base class's members
  - Most game studios prohibit or severely limit the use of multiple inheritance in their class hierarchies

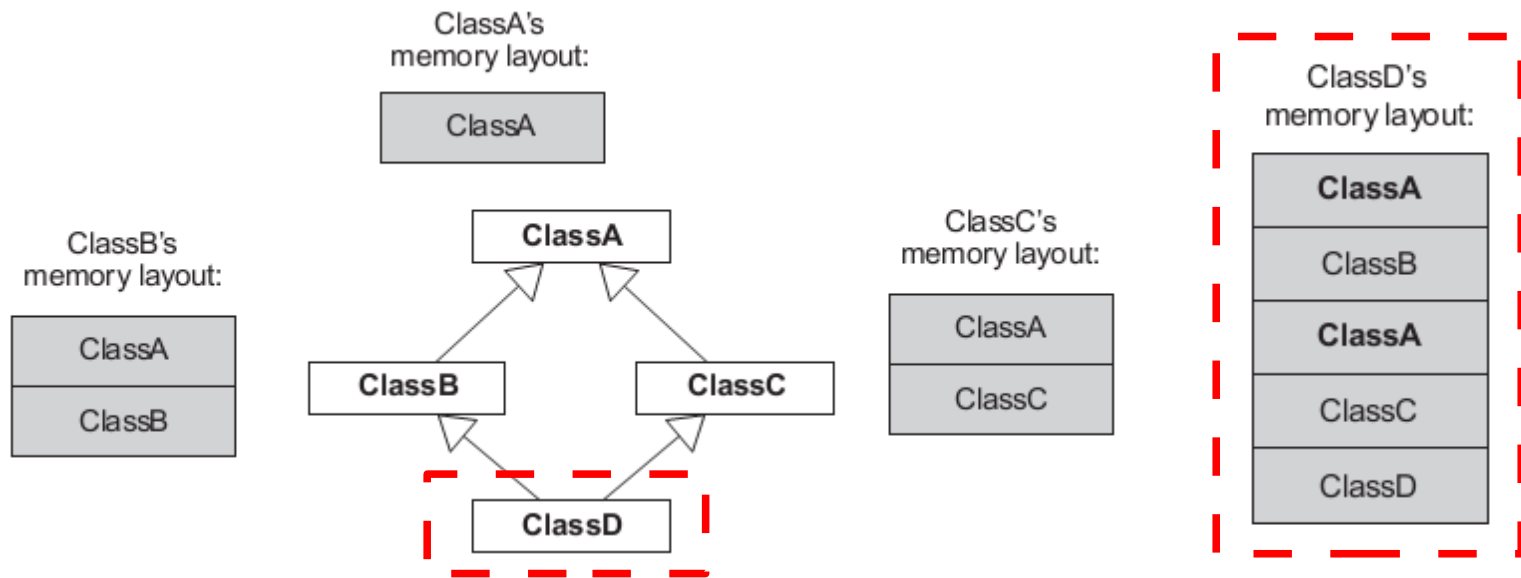


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

# Object-Centric Architectures (5/10)



- Mix-In Classes
  - A class may have multiple parent classes but only one grandparent
  - It may inherit from any number of mix-in classes (stand-alone classes with no base class)
  - Common functionality can be factored out into a mix-in class and then spot-patched into the main hierarchy

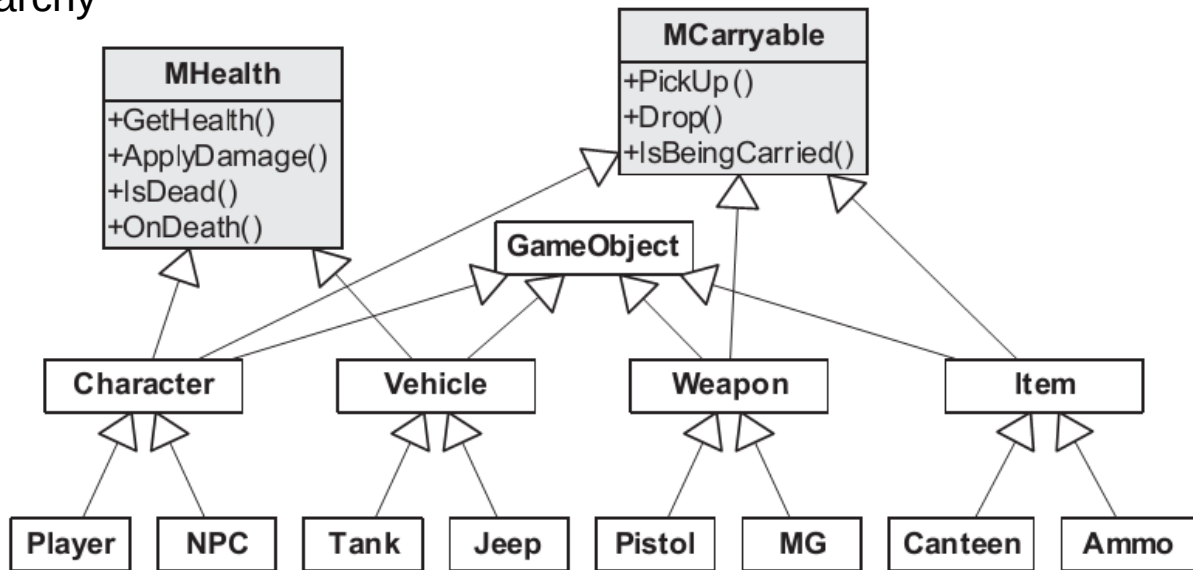


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

# Object-Centric Architectures (6/10)

---



- The Bubble-Up Effect
  - At first, the root classes are usually very simple, each one exposing only a minimal feature set
  - The desire to share code between two or more unrelated classes begins to cause features to “bubble up”
    - “can be carried”, “can float”, ...
  - Moving a feature into a base class that is common to all objects that need it
  - Possibly introducing additional member variables to allow distinction
    - `bool m_bCanBeCarried; bool m_bCanFloat; ...`
  - The Actor class in Unreal is a classic example
    - It contains data members and code for managing rendering, animation, physics, world interaction, audio effects, network replication for multiplayer games, object creation and destruction, actor iteration, and message broadcasting.
  - Makes encapsulating the functionality of various engine subsystems very difficult

Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

---

# Object-Centric Architectures (7/10)



- Using Composition to Simplify the Hierarchy
  - Inheritance → “is-a” relationship
  - Composition → “has-a” relationship
    - Composition → Class A owns an instance of Class B
    - Aggregation → Class A references Class B but doesn't manage its lifetime
- Problems with hierarchical Game Object structures
  - Our design choices when creating new types of game objects are limited
    - Every PhysicalObject is automatically a RenderableObject and an AnimatingObject even if that is not necessary
  - It is difficult to extend the functionality of the existing classes
    - What if we need a new types of AnimatingObjects like SkeletalObject and MorphObject, which also needs to be PhysicalObjects?

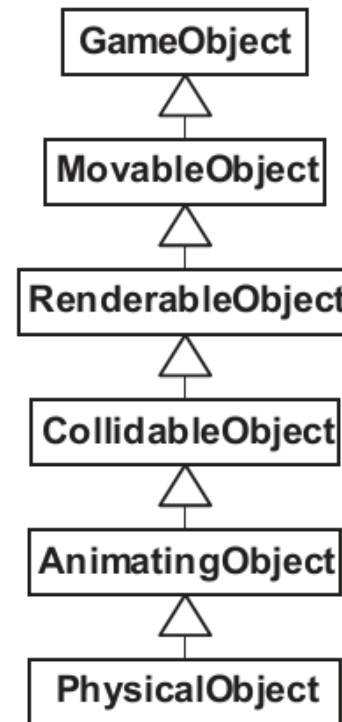


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014



# Object-Centric Architectures (8/10)



- Re-factored class hierarchy to favour composition over inheritance
  - Isolate the various features of a GameObject into independent classes (called components or service objects)
  - We can select only those features we need for each type of game object
  - Components are decoupled from one another
    - Allows subsystems to remain distinct and well-encapsulated

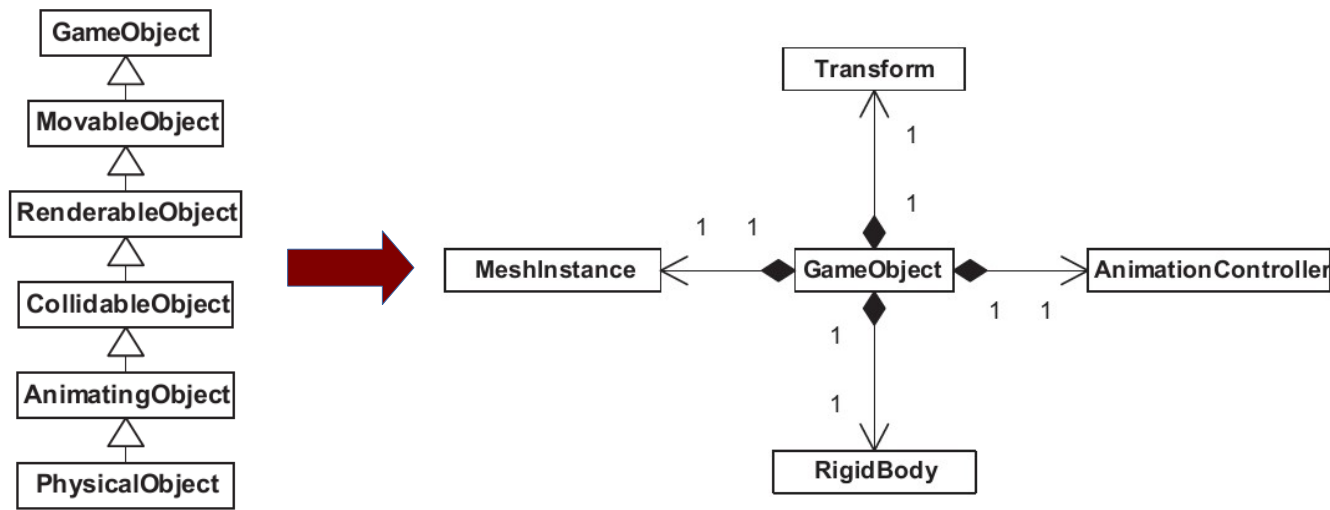


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

# Object-Centric Architectures (9/10)



- Component Creation and Ownership (1)
  - Simple Approach: The GameObject class has pointers to all possible components
    - Very easy to implement
    - Somewhat inflexible (e.g. adding new component types)
- Generic Components
  - Trickier to implement (GameObject must be completely generic)
  - New components can be created without modifying the GameObject class

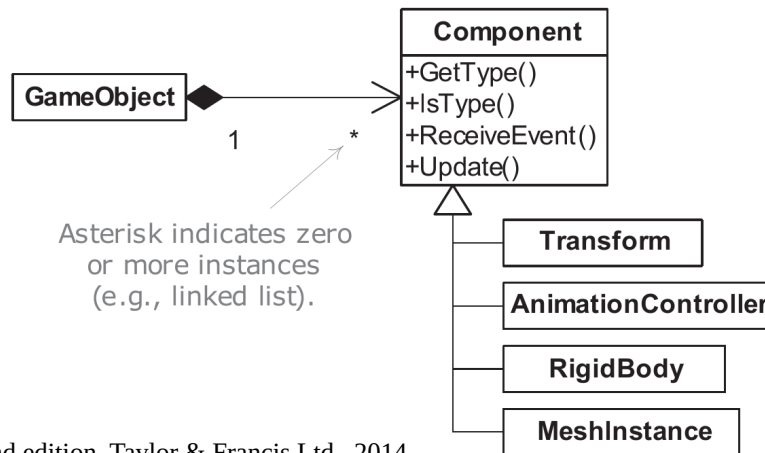


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

# Object-Centric Architectures (10/10)



- Component Creation and Ownership (2)
  - Pure Component Models
    - GameObject class would be a behaviour-less container (or non-existent)
    - The components are linked together into a logical grouping by id
  - Problems:
    - Definition of concrete GameObjects (probably using a factory)
    - Inter-component communication (no GameObject as “hub”)

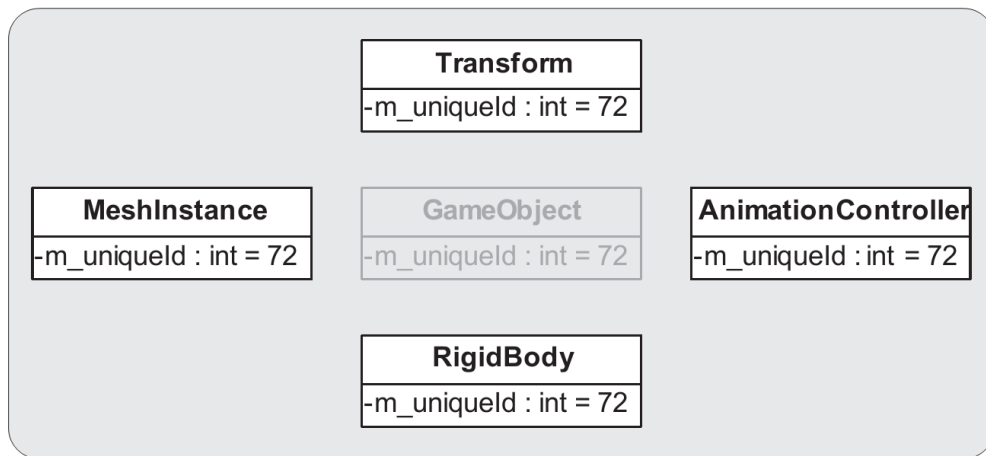


Image sources: Game Engine Architecture, 2nd edition, Taylor & Francis Ltd., 2014

# Property-Centric Architectures (1/2)



- Think primarily in terms of the attributes, rather than the objects
  - Define the set of all properties that a game object might have
  - Build a table containing the values of that property corresponding to each game object that has it
  - Property values are keyed by the objects' unique ids
  - A property-centric design is more akin to a relational database than an object model

## **Object1**

Position = (0, 3, 15)  
Orientation = (0, 43, 0)

## **Object2**

Position = (-12, 0, 8)  
Health = 15

## **Object3**

Orientation = (0, -87, 10)



## **Position**

Object1 = (0, 3, 15)  
Object2 = (-12, 0, 8)

## **Orientation**

Object1 = (0, 43, 0)  
Object3 = (0, -87, 10)

## **Health**

Object2 = 15

# Property-Centric Architectures (2/2)

---



- An object is defined not only by its attributes, but also by its behavior
- If all we have are tables of properties, then where do we implement the behavior?
  - Implementing behavior via property classes
    - Each property class can provide behaviors via its hard-coded methods (member functions).
    - The overall behavior of a particular game object is determined by the aggregation of the behaviors of all its properties.
  - Implementing behavior via script
    - A block of script code can be defined for every game object that will manage the object's behavior
  - Both methods can be combined
    - Hard-coded property classes provide basic behaviors
    - A facility is provided allowing game designers and programmers to implement new property types entirely in script

# Properties vs. Components

---



- Property objects are very closely related to components
  - A single logical game object is made up of multiple sub-objects
  - The main distinction lies in the roles of the sub-objects
- Property-centric design
  - Each sub-object defines a particular attribute of the game object itself
    - E.g., health, visual representation, inventory, a particular magic power, etc.
- Component-based (object-centric) design
  - The sub-objects often represent linkages to particular low-level engine subsystems
    - E.g. renderer, animation, collision and dynamics, etc.

# Pros and Cons of Property-Centric Designs

---

- Pros
  - Tends to be more memory efficient, because we need only store attribute data that is actually in use
  - It's easier to construct such a model in a data-driven manner
  - Can be more cache-friendly than an object-centric model, because data of the same type is stored contiguously in memory
    - struct of arrays vs. array of structs
- Cons
  - It's more difficult to enforce relationships between properties
  - It can be hard to implement a desired large-scale behavior merely by cobbling together the fine-grained behaviors of a group of property objects.
  - It's trickier to debug such systems
    - A game object's properties can not be inspected all at once in the watch window of the debugger

# High-performance object models in practice



- Data Oriented Design (DOD)
  - Motivated by efficient usage of the CPU cache
  - Focus is on the data layout
    - separating and sorting fields
    - think about *transformations* of data
  - Functions that *transform* the data
  - Procedural style of programming
- Entity Component Systems (ECS)
  - Entities with components of data
  - Favors composition over inheritance
  - Entity is defined not by a type, but by the components that are associated with it
  - Often combined with job systems

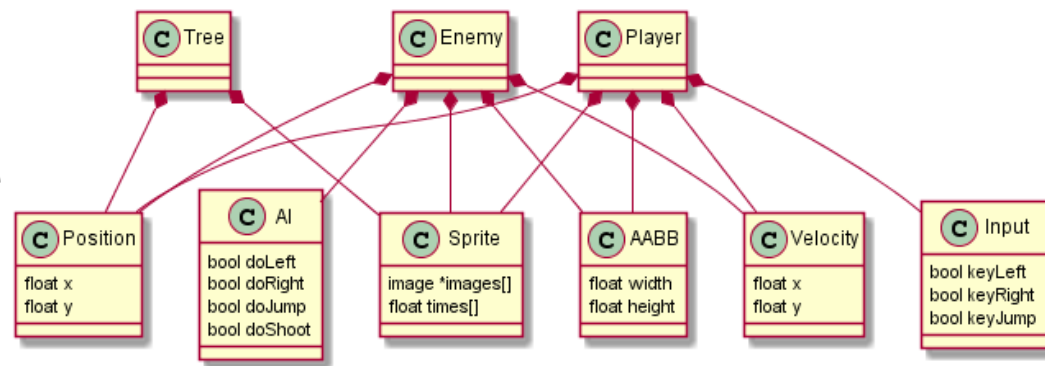
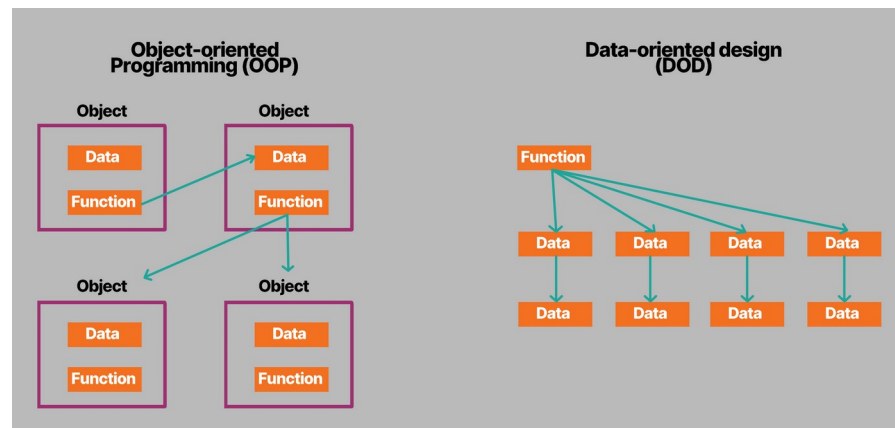


Image sources: <https://learn.unity.com/tutorial/what-is-dots-and-why-is-it-important?signup=true#5efa004cedbc2a021709de56>, <https://www.gamedev.net/tutorials/programming/general-and-gameplay-programming/understanding-component-entity-systems-r3013/>



# Special Purpose Game Objects (1/2)

---



- Game Object may be invisible, but can still be interacted with
  - Trigger, zones, portals, ...
  - Provide interactivity for static objects
- Game Objects may be completely hidden
  - Timer or clocks
  - Transformations (NullObjects), splines, paths, ...
- The camera often is a special kind of game object as well
- Depending on the engine's software design, literally everything can be implemented as a game object!
  - There are many more game objects in addition to the plain obvious ones

Image sources: -

# Special Purpose Game Objects (2/2)



- Example: Der Fluch der Osterinsel



# Game Engine Architecture: Bill Clark (CodeLabs Tech Talk 2020)

---



<https://youtu.be/mUeNqLcx4eI?t=3144>

Image sources: <https://learn.unity.com/tutorial/entity-component-system#5c7f8528edbc2a002053b676>

---

# Conclusions

---



- Game objects are the dynamic elements of a game
  - Their state changes over time
  - The game state is current state of all game objects, taken as a whole
  - There are usually much more game object than just the visible ones
- Game Objects are usually build of several components
  - Helps to maintain a clean and manageable software architecture
  - Makes creating different kinds of game objects very flexible
  - Can be centered around components or properties
  - Encapsulation mechanism for engine subsystems
  - Performance benefit
    - “struct of arrays” is more chache efficient than “array of structs”